

Exercises for Week 10

1. GPU Reductions

This exercise is based on the excellent presentation of [parallel reductions in CUDA](#) by Mark Harris. While it's a bit dated by now, it still contains many pearls of wisdom regarding high performance CUDA code.

Consider the following implementation of a parallel reduction on a GPU:

```
1 from numba import cuda
2
3 TPB = 128 # Threads per block
4
5 @cuda.jit
6 def reduce_kernel(data, out, n):
7     # Get the 1D grid and block indices
8     tid = cuda.threadIdx.x
9     i = cuda.grid(1)
10
11     # Do reduction for threadblock
12     s = 1
13     while s < cuda.blockDim.x:
14         if tid % (2 * s) == 0 and i + s < n:
15             data[i] += data[i + s]
16         s *= 2
17         cuda.syncthreads() # Ensure block is synchronized
18
19     # Write result for this block to global memory
20     if tid == 0:
21         out[cuda.blockIdx.x] = data[i]
22
23 def get_grid(n, tpb):
24     return (n + (tpb - 1)) // tpb # Blocks per grid
25
26 def reduce(x):
27     n = len(x)
28     bpg = get_grid(n, TPB)
29     out = cuda.device_array(bpg, dtype=x.dtype)
30     while bpg > 1:
31         reduce_kernel[bpg, TPB](x, out, n)
32         n = bpg
33         bpg = get_grid(n, TPB)
34         x[:n] = out[:n]
35     reduce_kernel[bpg, TPB](x, out, n)
36     return out
```

1. **Autolab** Make a Python program that takes a number n as a command line argument. The program must then generate n random float32 numbers and use the above code to compute their sum. Use `np.random.rand` to generate the random numbers and convert them to float32 with `np.astype`. Finally, print the sum (and only the sum).
2. For now, do *not* manually transfer the data to the GPU - let Numba do it when the kernel is called. Run your program for $n = 4000000$ under the `nsys` profiler and view the statistics. Where is time spent?
Hint: run `nsys profile -o <prof_data_file> python program.py <args>`.
To view statistics: `nsys stats <prof_data_file>.nsys-rep`.
3. Modify your program such that you manually transfer the data to the GPU to avoid wasted transfers. Re-run your program under the `nsys` profiler. What has changed?
4. Modify the kernel such that each thread block loads its part of the data into a shared memory array. Then, do the reduction in the shared array instead of in the global data array. Note: this will not have a large effect on speed now, but will make the next optimization simpler (less book keeping) and faster.
5. A major reason the current kernel is slow is the `if tid % (s * 2) == 0` condition. This causes a lot of warp divergence which slows the kernel down. Instead of "masking out" threads, change the kernel to use a strided index as shown on [slide 11 in the linked presentation by Mark Harris](#).
6. Profile the optimized kernel. What has changed?
7. (Optional) These exercises were only the first few optimizations presented in the above reference. There is still plenty of performance to be squeezed out! Implement some (or all) of the remaining optimizations. The code in the presentation is in C++, but everything translates neatly to Numba functions.

Note also, that modern CUDA versions have introduced helper functions that can now make reductions even faster. For more info on this, see [this technical blog from NVIDIA](#).

Hint: you may need to apply the reduction to a larger set of numbers to really see speed benefits.

2. CuPy

In this exercise, we will use CuPy to speed up existing NumPy code - specifically, the haversine distance code we worked with in week 4. The single loop and no loop distance matrix implementations were:

```
1 def distance_matrix_one_loop(p1, p2):
2     p1 = np.radians(p1)
3     p2 = np.radians(p2)
4
```

```

5     D = np.empty((len(p1), len(p2)))
6     for i in range(len(p1)):
7         dsin2 = np.sin(0.5 * (p1[i] - p2)) ** 2
8         cosprod = np.cos(p1[i, 0]) * np.cos(p2[:, 0])
9         a = dsin2[:, 0] + cosprod * dsin2[:, 1]
10        row = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
11        D[i, :] = row
12
13    D *= 6371 # Earth radius in km
14    return D
15
16 def distance_matrix_noloop(p1, p2):
17     p1 = np.radians(p1)
18     p2 = np.radians(p2)
19     dsin2 = np.sin(0.5 * (p1[:, None, :] - p2[None, :, :])) ** 2
20     cosprod = np.cos(p1[:, None, 0]) * np.cos(p2[None, :, 0])
21     D = 2 * np.arcsin(np.sqrt(dsin2[:, :, 0] + cosprod * dsin2[:, :,
22     1]))
23     D *= 6371 # Earth radius in km
24     return D

```

For reference, the location data was stored at:

/dtu/projects/02613_2025/data/locations/

1. **Autolab** Convert both implementations such that they use CuPy operations instead of NumPy. Also modify your Python program from week 4 that used the distance matrix functions to use CuPy.
2. Measure the run time for your CuPy program using the single loop version. Use the subset with 5000 rows `locations_5000.csv` file. How does the run time compare to the CPU version from week 4? You may need to run the programs a few times to get stable times. For simplicity, you may simply measure the run time with the `time` command: `time python points.py <path to data>`
3. Re-run the program under the `nsys` profiler. Where does the program spend its time? Be sure to check all sections (except the OS Runtime Summary) to get a full overview.
4. Measure the run time for your CuPy program using the no loop version. What is the new run time?
5. Re-run the program under the `nsys` profiler. Where does the program now spend its time? What has changed compared to the single loop version? Be sure to check all sections to get a full overview.